

CẨM nang kiến thức đồ án

Phân loại Tồn thương Đa bằng Học sâu

Từ định nghĩa cơ bản đến chi tiết kỹ thuật

Nguyễn Trọng Bách — 23BI14057

Đại học Khoa học và Công nghệ Hà Nội (USTH)

Mục lục

1	Đồ án này là gì — tóm tắt một trang	4
2	Các định nghĩa nền tảng	4
2.1	Học sâu (Deep Learning) là gì?	4
2.2	Pretrain (Pre-training)	4
2.3	Fine-tune (Fine-tuning)	5
2.4	Transfer Learning	5
2.5	Đồ án này dùng pretrain hay fine-tune?	5
3	Dữ liệu ISIC 2018	6
3.1	Bộ dữ liệu HAM10000	6
3.2	Mất cân bằng dữ liệu — vấn đề trung tâm	7
3.3	Chia tập theo Stratified Sampling 70/15/15	7
4	Tiền xử lý và Augmentation	7
4.1	Pipeline tổng quan	8
4.2	Tiền xử lý cơ bản	8
4.3	Augmentation cho train set	9
5	Weighted Random Sampler	9
5.1	Ý tưởng	9
5.2	Công thức	9
5.3	Code	10
6	Mixup	10
6.1	Ý tưởng	10
6.2	Công thức	10
6.3	Ví dụ cụ thể	11
6.4	Code	11
6.5	Tại sao Mixup quan trọng?	11

7	CutMix	11
7.1	Ý tưởng	12
7.2	Công thức	12
7.3	Ví dụ cụ thể	12
7.4	Code	12
7.5	So sánh Mixup vs. CutMix	13
8	Label Smoothing	13
8.1	Ý tưởng	13
8.2	Công thức	13
8.3	Ví dụ	13
8.4	Tại sao hiệu quả	13
8.5	Code	13
9	Bốn kiến trúc mô hình được so sánh	14
9.1	EfficientNet-B0 (5.3M tham số)	14
9.2	ResNet50 (25.6M tham số)	14
9.3	DenseNet121 (8.0M tham số)	14
9.4	Swin-Tiny (28.0M tham số)	14
9.5	Tóm tắt	14
10	Huấn luyện	14
10.1	Hàm mất mát	15
10.2	Optimizer: AdamW	15
10.3	Scheduler: Cosine Annealing với Warmup	15
10.4	Siêu tham số	15
10.5	Vòng lặp huấn luyện	15
11	Ensemble bằng Soft Voting	16
11.1	Tại sao cần ensemble?	16
11.2	Soft voting — công thức	16
11.3	Ví dụ cụ thể	17
11.4	Tại sao soft voting hiệu quả? — Phân rã bias–variance	17
11.5	Code	17
11.6	So sánh với phương án khác	18
12	Grad-CAM — khả diễn giải	18
12.1	Tại sao Grad-CAM xài được?	18
12.2	Bốn bước toán học	18

12.3	Trực giác	19
12.4	Code GradCAM class	19
12.5	Target layer mỗi mô hình khác nhau — tại sao?	20
12.6	Grad-CAM định lượng	20
13	Các chỉ số đánh giá	21
13.1	Tại sao Accuracy không đủ?	21
13.2	Định nghĩa từng chỉ số	21
13.3	Ví dụ tính F1, Precision, Recall	21
13.4	Tại sao Sensitivity cho MEL được ưu tiên?	21
14	Ablation Study	21
14.1	Phương pháp	21
14.2	Kết quả	22
14.3	Diễn giải từng thành phần	22
15	Kết quả thực nghiệm chính	22
15.1	Bảng so sánh 4 mô hình đơn + Ensemble	22
15.2	Độ tin cậy thống kê (multi-seed)	22
15.3	Hiệu năng per-class của Ensemble	23
16	Code cho các biểu đồ trong report	24
16.1	Confusion Matrix	24
16.2	ROC Curves (One-vs-Rest)	25
16.3	Training History (loss, accuracy, F1, LR)	25
16.4	Per-class F1 bar chart	25
16.5	Grad-CAM overlay	26
17	So sánh với State-of-the-Art	26
18	Hạn chế và hướng phát triển	26
18.1	Hạn chế	27
18.2	Hướng phát triển	27
19	Tổng kết một trang	27

1. Đề án này là gì — tóm tắt một trang

Bài toán. Cho một ảnh dermoscopy (ảnh soi da bằng kính phóng đại) của một tổn thương trên da, hãy dự đoán nó thuộc một trong **7 loại**: akiec, bcc, bk1, df, mel, nv, vasc. Trong đó mel (melanoma — ung thư hắc tố) là lớp nguy hiểm nhất và là mục tiêu phát hiện hàng đầu.

Dữ liệu. ISIC 2018 Task 3 / HAM10000 — 10.015 ảnh, mất cân bằng nghiêm trọng (lớp nv chiếm 67%, lớp df chỉ 1.1%, tỉ lệ NV:DF $\approx$ 58:1).

Hướng tiếp cận. Fine-tune bốn kiến trúc học sâu pretrained trên ImageNet (*EfficientNet-B0*, *ResNet50*, *DenseNet121*, *Swin-Tiny*) với pipeline điều chuẩn để xử lý mất cân bằng: Weighted Random Sampler, Mixup, CutMix, Label Smoothing. Sau đó kết hợp bốn mô hình bằng **soft-voting ensemble**.

Kết quả.

- Mô hình đơn tốt nhất (hạt giống đại diện): EfficientNet-B0 — Accuracy 88.36%, Macro F1 0.831.
- Ensemble** (soft voting 4 mô hình, hạt giống đại diện): Accuracy 89.49%, Macro F1 0.845, AUC 0.980.
- Ensemble trung bình qua 5 hạt giống ngẫu nhiên**: $89,93 \pm 0,66\%$ Accuracy, $0,852 \pm 0,010$ Macro F1, $0,982 \pm 0,001$ Macro AUC. McNemar test xác nhận ensemble vượt mọi mô hình đơn có ý nghĩa thống kê ($p < 10^{-7}$ so với mô hình mạnh nhất Swin-Tiny).

Khả diễn giải. Grad-CAM (Selvaraju et al., 2017) tạo bản đồ nhiệt cho biết *vùng nào trên ảnh đã ảnh hưởng đến quyết định của mô hình*. Bằng chứng định lượng: `focus_ratio` > 0.6 trên cả 4 mô hình — mô hình thật sự nhìn vào tổn thương chứ không phải nhiễu (lông, bọt gel).

Sản phẩm. Một prototype Streamlit (`app.py`) cho phép bác sĩ upload ảnh và xem dự đoán + Grad-CAM trực tiếp.

Thông điệp chính của đề án

Một pipeline được điều chuẩn cẩn thận có thể thu hẹp phần lớn khoảng cách giữa mô hình nhẹ và mô hình lớn trên tập dữ liệu y tế hạn chế. Đóng góp lớn nhất không đến từ kiến trúc mạnh, mà từ các thành phần **regularization** (Mixup/CutMix, Label Smoothing) cộng với **ensemble** có chủ đích.

2. Các định nghĩa nền tảng

2.1. Học sâu (Deep Learning) là gì?

Học sâu là một nhánh của *học máy* trong đó mô hình được tổ chức thành nhiều **lớp (layer)** xếp chồng. Mỗi lớp biến đổi đầu vào thành biểu diễn trừu tượng hơn. Với ảnh, các lớp đầu học *cạnh*, *góc*, *texture*; các lớp giữa học *bộ phận* (một mảng sắc tố nâu); các lớp cuối học *khái niệm* (đây là một nốt ruồi vs. một khối melanoma).

Mạng học bằng cách **cực tiểu hóa hàm mất mát** qua thuật toán *gradient descent*. Cho dữ liệu (x_i, y_i) và tham số θ :

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i).$$

2.2. Pretrain (Pre-training)

Pretrain = huấn luyện mô hình *từ đầu* (random initialization) trên một tập dữ liệu **lớn và tổng quát**, mục đích là học các đặc trưng chung của loại dữ liệu đó.

Pretrain trên ImageNet

ImageNet-1K có **1.28 triệu** ảnh thuộc **1000 lớp** (chó, mèo, xe hơi, hoa, công cụ, ...). Huấn luyện EfficientNet-B0 trên ImageNet mất hàng tuần trên nhiều GPU. Kết quả là một file weights θ_{pre} — mô hình *đã biết nhìn ảnh*, đã học cạnh, texture, hình dạng, dù chưa biết gì về da liễu.

2.3. Fine-tune (Fine-tuning)

Fine-tune = lấy mô hình *đã được pretrain*, sửa đầu ra cho phù hợp bài toán mới, rồi huấn luyện tiếp toàn bộ (hoặc một phần) mô hình trên tập dữ liệu nhỏ hơn và chuyên biệt hơn.

Fine-tune EfficientNet-B0 cho ISIC 2018

1. Tải weights θ_{pre} (ImageNet pretrained) từ thư viện `timm`.
2. Thay classifier head: `Linear(1280, 1000) → Linear(1280, 7)` (khởi tạo ngẫu nhiên).
3. Huấn luyện toàn bộ mô hình trên 7.010 ảnh train ISIC với learning rate nhỏ (3×10^{-4}) trong 50 epoch.

2.4. Transfer Learning

Transfer Learning = chuyển giao tri thức từ một bài toán nguồn (ImageNet) sang bài toán đích (ISIC 2018). Pretrain + Fine-tune là cặp công cụ phổ biến nhất của transfer learning. Lý do hiệu quả: các lớp conv đầu tiên đã học *cạnh, texture, hình dạng* — những features tổng quát có ích cho *mọi* bài toán nhận dạng ảnh, bao gồm cả ảnh dermoscopy.

	Train từ đầu	Fine-tune
Dữ liệu cần	>100.000 ảnh	vài nghìn ảnh
Thời gian	Hàng tuần	Vài chục phút
Kết quả trên data nhỏ	Kém (overfit nặng)	Tốt hơn rõ rệt
Lý do	Học features từ scratch	Tận dụng features ImageNet

2.5. Đồ án này dùng pretrain hay fine-tune?

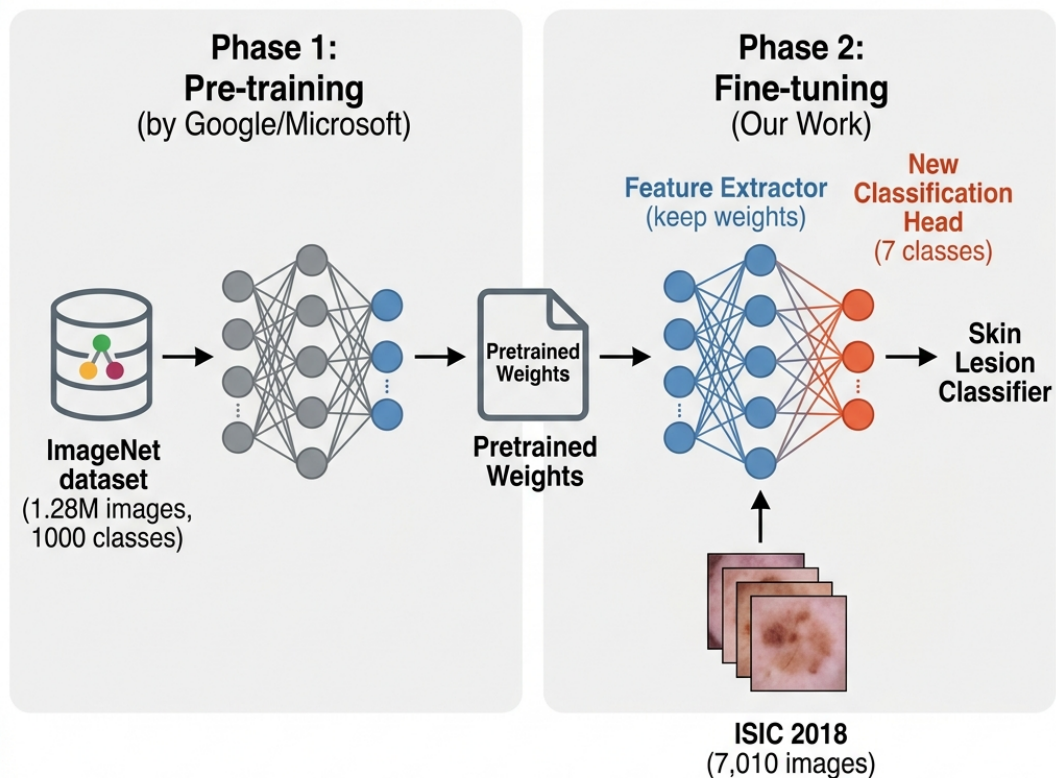
Đồ án dùng fine-tune. Không tự pretrain (vì chỉ có 10K ảnh, không đủ). Lấy weights ImageNet sẵn từ `timm` (`pretrained=True`) rồi fine-tune trên ISIC 2018.

Listing 1: `src/models/build_model.py` — fine-tune từ ImageNet

```
1 import timm
2
3 def build_model(model_cfg: dict):
4     model = timm.create_model(
5         model_cfg['name'],          # "tf_efficientnet_b0", "resnet50", ...
6         pretrained=True,           # Load ImageNet weights
7         num_classes=7,             # thay classifier head: 1000 → 7
8         drop_rate=0.3,             # dropout ướtrc classifier
9     )
10    return model
```

Cờ `pretrained=True` làm hai việc: (i) tải file weights ImageNet về cache, (ii) thay `Linear(d, 1000)` bằng `Linear(d, 7)` mới khởi tạo random.

Transfer Learning and Fine-tuning



Sơ đồ transfer learning: pretrain trên ImageNet → giữ feature extractor, thay classifier head, fine-tune trên ISIC 2018.

3. Dữ liệu ISIC 2018

3.1. Bộ dữ liệu HAM10000

ISIC 2018 Task 3 (tên khác: HAM10000) là benchmark chuẩn cho phân loại tổn thương da. Gồm **10.015** ảnh dermoscopy chia thành 7 lớp:

Mã	Tên đầy đủ	Train	Val	Test	Tổng
nv	Melanocytic Nevus (nốt ruồi)	4.693	1.006	1.006	6.705
mel	Melanoma (ung thư hắc tố)	779	167	167	1.113
bkl	Benign Keratosis-like	769	165	165	1.099
bcc	Basal Cell Carcinoma (ung thư biểu mô)	360	77	77	514
akiec	Actinic Keratosis (tiền ung thư)	229	49	49	327
vasc	Vascular Lesion (tổn thương mạch máu)	99	21	22	142
df	Dermatofibroma (u xơ)	81	17	17	115
Tổng		7.010	1.502	1.503	10.015

3.2. Mất cân bằng dữ liệu — vấn đề trung tâm

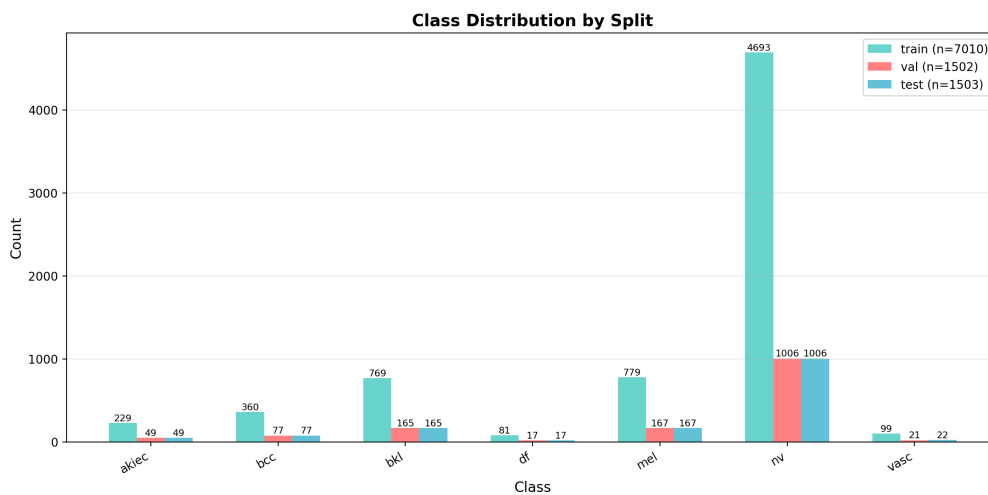
Tỉ lệ lớp đông nhất (nv: 4.693) so với lớp ít nhất (df: 81) là **58:1**. Đây là *vấn đề lâm sàng kinh điển* của mọi bài toán y tế: bệnh lý nguy hiểm như melanoma thường **hiếm** so với tổn thương lành tính.

Tại sao mất cân bằng là vấn đề lớn?

Mạng nơ-ron tối thiểu hóa loss **trung bình**, nên ưu tiên lớp đông. Nếu không xử lý, mô hình có thể đạt 67% Accuracy chỉ bằng cách dự đoán "nv" cho mọi ảnh — nhưng **vô dụng lâm sàng** vì sẽ bỏ sót 100% melanoma. Vì vậy ta dùng **Macro F1** làm metric chính: nó tính F1 trung bình *không trọng số* của 7 lớp, ép mô hình quan tâm đến cả lớp hiếm.

3.3. Chia tập theo Stratified Sampling 70/15/15

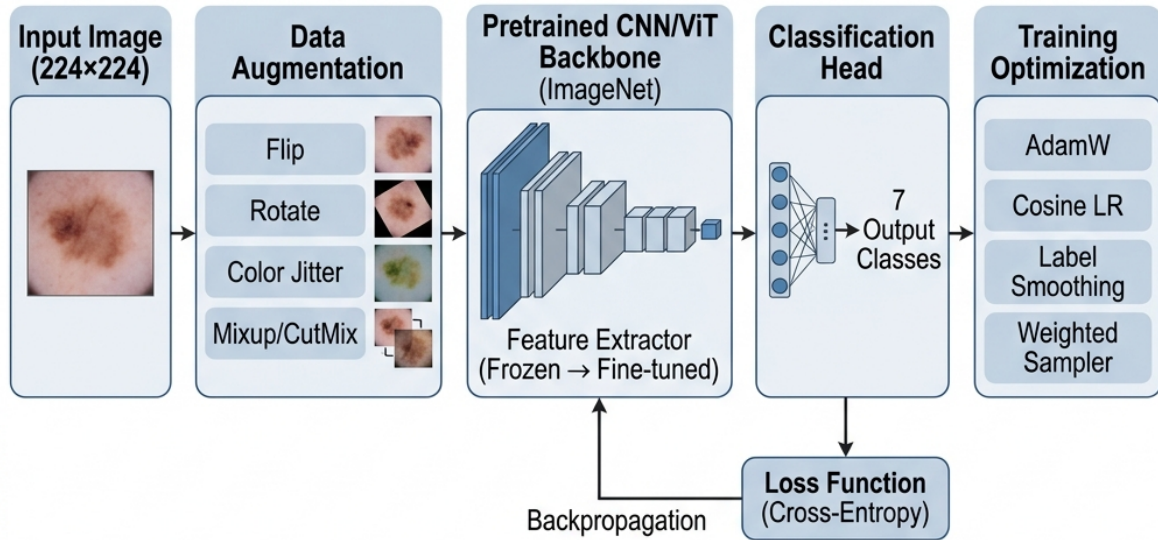
Tập gốc được chia tỉ lệ **70% train / 15% val / 15% test** bằng *stratified sampling*: mỗi split giữ nguyên tỉ lệ 7 lớp. Quan trọng để val/test có cùng phân phối với train — nếu không, kết quả đo được sẽ không đáng tin.



Phân bố 7 lớp giữ nguyên qua train/val/test (stratified split).

4. Tiền xử lý và Augmentation

4.1. Pipeline tổng quan



Toàn bộ pipeline: ảnh thô $\rightarrow$ augmentation $\rightarrow$ backbone pretrained $\rightarrow$ classifier head $\rightarrow$ loss.

4.2. Tiền xử lý cơ bản

Mọi ảnh đều được:

- **Resize** về 224×224 (kích thước input chuẩn cho ImageNet pretrained).
- **Chuyển sang tensor** (ToTensor) — giá trị pixel $[0, 255] \rightarrow [0, 1]$.
- **Normalize** bằng mean/std của ImageNet:

$$x' = \frac{x - \mu_{\text{ImageNet}}}{\sigma_{\text{ImageNet}}}, \quad \mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225).$$

Lý do dùng mean/std của ImageNet: mô hình pretrained ”kỳ vọng” input ở thang đo đó. Nếu normalize sai, distribution input lệch khỏi training distribution của ImageNet, hiệu năng tụt.

4.3. Augmentation cho train set

Augmentation = biến đổi ngẫu nhiên ảnh train mỗi epoch để mô hình thấy nhiều ”biến thể”, giảm overfit. **Chỉ áp dụng trên train, không áp dụng trên val/test** (vì val/test cần phản ánh phân phối thực).

Phép biến đổi	Tham số	Lý do
RandomResizedCrop	scale 0.85–1.0	Tăng scale invariance
HorizontalFlip	$p = 0.5$	Ảnh da đối xứng theo trục
VerticalFlip	$p = 0.5$	Ảnh dermoscopy không có ”lên/xuống” cố định
Rotation	$\pm 90^\circ$	Bất biến theo xoay
ColorJitter	b/c/s/h = 0.25	Robust với khác biệt thiết bị
GaussianBlur	$p = 0.1$	Robust với độ nét khác nhau
RandomErasing	$p = 0.1$	Robust với che khuất

Listing 2: src/data/transforms.py — pipeline augmentation

```
1 def build_transforms(image_size, aug_cfg, is_train):
2     normalize = transforms.Normalize(
3         mean=[0.485, 0.456, 0.406],
4         std=[0.229, 0.224, 0.225],
5     )
6     if is_train:
7         train_tfms = [
8             transforms.RandomResizedCrop(image_size, scale=(0.85, 1.0)),
9             transforms.RandomHorizontalFlip(p=0.5),
10            transforms.RandomVerticalFlip(p=0.5),
11            transforms.RandomRotation(degrees=90),
12            transforms.ColorJitter(brightness=0.25, contrast=0.25,
13                                   saturation=0.25, hue=0.1),
14            transforms.RandomApply([
15                transforms.GaussianBlur(kernel_size=5, sigma=(0.1, 2.0)),
16            ], p=0.1),
17            transforms.ToTensor(),
18            normalize,
19            transforms.RandomErasing(p=0.1),
20        ]
21        return transforms.Compose(train_tfms)
22    # Eval: chỉ resize + normalize
23    return transforms.Compose([
24        transforms.Resize((image_size, image_size)),
25        transforms.ToTensor(), normalize,
26    ])
```

5. Weighted Random Sampler

5.1. Ý tưởng

Thay vì để DataLoader chọn ảnh đều nhau, ta gán **trọng số** cho mỗi mẫu sao cho lớp hiếm được chọn thường xuyên hơn. Mỗi mini-batch khi đó xấp xỉ cân bằng 7 lớp.

5.2. Công thức

Cho N = tổng mẫu train, $C = 7$, n_c = số mẫu lớp c . Trọng số mỗi mẫu thuộc lớp c :

$$w_c = \frac{1}{n_c} \quad (\text{hoặc dạng chuẩn hóa}) \quad w_c = \frac{N}{C \cdot n_c}.$$

Tính trọng số cụ thể trên ISIC 2018 train

$N = 7.010$, $C = 7$. Bảng trọng số 7 lớp:

Lớp	n_c	$w_c = N / (C \cdot n_c)$
nv	4.693	0.213
mel	779	1.286
bkl	769	1.303
bcc	360	2.782
akiec	229	4.373
vasc	99	10.117
df	81	12.365

Hệ quả: một ảnh df được DataLoader chọn với xác suất cao gấp $12.365/0.213 \approx 58\times$ so với một ảnh nv. Sau khi sampling, mini-batch xấp xỉ cân bằng.

5.3. Code

Listing 3: src/data/samplers.py

```
1 import numpy as np
2 import torch
3 from torch.utils.data import WeightedRandomSampler
4
5 def build_weighted_sampler(labels, num_classes):
6     counts = np.bincount(labels, minlength=num_classes).astype(float)
7     counts[counts == 0] = 1.0
8     class_weights = 1.0 / counts # w_c
9     sample_weights = [class_weights[lbl] for lbl in labels]
10    sample_weights = torch.as_tensor(sample_weights, dtype=torch.double)
11    return WeightedRandomSampler(
12        weights=sample_weights,
13        num_samples=len(sample_weights),
14        replacement=True,
15    )
```

6. Mixup

6.1. Ý tưởng

Mixup (Zhang et al., ICLR 2018) trộn *tuyến tính* hai ảnh và hai nhãn của chúng để tạo một ảnh huấn luyện "ảo". Buộc mô hình hoạt động *linearly* giữa các mẫu — một dạng regularization rất mạnh.

6.2. Công thức

Cho hai mẫu (x_i, y_i) và (x_j, y_j) . Lấy $\lambda \sim \text{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j.$$

Trong đồ án $\alpha = 0.2 \Rightarrow \lambda$ thường gần 0 hoặc 1, ít khi ≈ 0.5 . Nghĩa là Mixup pha trộn nhẹ, không phá ảnh quá mức.

6.3. Ví dụ cụ thể

Mixup giữa 1 ảnh melanoma và 1 ảnh nevus

- x_i = ảnh melanoma, $y_i = \text{mel}$ (one-hot $[0, 0, 0, 0, 1, 0, 0]$)
- x_j = ảnh nốt ruồi, $y_j = \text{nv}$ (one-hot $[0, 0, 0, 0, 0, 1, 0]$)
- Lấy $\lambda = 0.7$ ngẫu nhiên từ $\text{Beta}(0.2, 0.2)$

Ảnh trộn: $\tilde{x} = 0.7 x_i + 0.3 x_j$ — ảnh melanoma ”rõ” 70%, ảnh nốt ruồi ”mờ” 30% chồng lên.

Nhãn trộn: $\tilde{y} = 0.7 \cdot [0, 0, 0, 0, 1, 0, 0] + 0.3 \cdot [0, 0, 0, 0, 0, 1, 0] = [0, 0, 0, 0, 0.7, 0.3, 0]$

Mô hình phải dự đoán xác suất MEL gần 0.7 và NV gần 0.3.

Loss cho nhãn soft. Cross-entropy thường dùng nhãn nguyên. Với nhãn lai, ta tính:

$$\mathcal{L} = \lambda \text{CE}(f_{\theta}(\tilde{x}), y_i) + (1 - \lambda) \text{CE}(f_{\theta}(\tilde{x}), y_j).$$

Tương đương loss với nhãn soft. Đây là logic của hàm `mixup_criterion`.

6.4. Code

Listing 4: `src/data/mixup.py` — hàm `mixup_data`

```
1 def mixup_data(images, targets, alpha=0.2):
2     lam = np.random.beta(alpha, alpha) if alpha > 0 else 1.0
3     batch_size = images.size(0)
4     index = torch.randperm(batch_size, device=images.device)
5
6     mixed_images = lam * images + (1 - lam) * images[index]
7     targets_a = targets
8     targets_b = targets[index]
9     return mixed_images, targets_a, targets_b, lam
10
11
12 def mixup_criterion(criterion, outputs, targets_a, targets_b, lam):
13     """Loss cho nhãn lai."""
14     return lam * criterion(outputs, targets_a) + \
15         (1 - lam) * criterion(outputs, targets_b)
```

6.5. Tại sao Mixup quan trọng?

- Với chỉ 81 ảnh df trong train, không có Mixup mô hình sẽ *ghi nhớ* các sample này trong vài epoch đầu.
- Mixup tạo ảnh mới mỗi batch, mô hình không thể ”memorize” được.
- Làm mịn ranh giới quyết định, tránh over-confidence.

Bằng chứng định lượng trong đồ án: bỏ Mixup/CutMix khỏi pipeline làm Macro F1 *giảm 4.51%* — đây là thành phần có đóng góp lớn nhất (xem §14).

7. CutMix

21
22

```
return mixed_images, targets, targets[index], lam
```

7.5. So sánh Mixup vs. CutMix

	Mixup	CutMix
Cách pha trộn	Tuyến tính toàn ảnh	Dán patch
Pixel gốc	Bị phá vỡ	Giữ nguyên trong patch
Tốt cho	Smoothing biên quyết định	Features cục bộ

Trong đồ án **áp dụng cả hai**, mỗi batch có xác suất 0.5 chọn 1 trong 2.

8. Label Smoothing

8.1. Ý tưởng

Mục tiêu của cross-entropy với nhãn one-hot là đẩy logit của lớp đúng về $+\infty$ và logit các lớp sai về $-\infty \Rightarrow$ *over-confidence*. **Label Smoothing** thay nhãn cứng bằng nhãn mềm, ép mô hình không "quá tự tin".

8.2. Công thức

$$y_k^{\text{LS}} = (1 - \varepsilon) \mathbb{I}[k = c] + \frac{\varepsilon}{C}, \quad \varepsilon = 0.1, C = 7.$$

8.3. Ví dụ

Smoothing nhãn melanoma

$y_{\text{onehot}} = [0, 0, 0, 0, 1, 0, 0]$ (lớp mel là 1, còn lại 0).

Với $\varepsilon = 0.1, C = 7$:

- Phần "đúng" giảm còn $1 - 0.1 = 0.9$
- Phần 0.1 được *rải đều* cho cả 7 lớp: mỗi lớp được $0.1/7 = 0.01429$

$$y^{\text{LS}} = [\mathbf{0.0143}, 0.0143, 0.0143, 0.0143, \mathbf{0.9143}, 0.0143, 0.0143].$$

Đúng kiểm tra: tổng $= 0.9143 + 6 \times 0.0143 = 1.000$. $\square$

8.4. Tại sao hiệu quả

- Không có Label Smoothing, model học đẩy logit lớp đúng $\rightarrow +\infty \Rightarrow$ vector logit có magnitude lớn không cần thiết.
- Smoothing đặt cận trên cho logit đúng và cận dưới cho logit sai $\Rightarrow$ *ranh giới quyết định mềm hơn*.
- Tốt cho các lớp tương đồng (MEL/NV/BKL) — ranh giới cứng sẽ sai nhiều ở vùng biên.

8.5. Code

PyTorch tích hợp sẵn:

Listing 6: src/models/losses.py

```

1 import torch.nn as nn
2
3 def build_loss(loss_cfg, class_weights=None, device='cpu'):
4     label_smoothing = float(loss_cfg.get('label_smoothing', 0.0))
5
6     if loss_cfg['name'] == 'cross_entropy':
7         return nn.CrossEntropyLoss(label_smoothing=label_smoothing)
8     ...

```

Một nghịch lý đã quan sát: bỏ Label Smoothing làm AUC *tăng* ($0.952 \rightarrow 0.967$) nhưng Macro F1 *giảm* 3.11%. Hard target $\Rightarrow$ model tự tin hơn (xếp hạng probability sắc nét $\Rightarrow$ AUC cao) nhưng ranh giới cứng làm sai ở vùng biên (F1 thấp).

9. Bốn kiến trúc mô hình được so sánh

9.1. EfficientNet-B0 (5.3M tham số)

Ý tưởng chính: *compound scaling* — đồng thời scale depth, width và resolution theo một hệ số chung. Sử dụng MBConv (Mobile Inverted Bottleneck) với *depthwise separable convolution* và *Squeeze-and-Excitation* (SE) để học global context theo kênh.

Feature map cuối: $(1280, 7, 7) \rightarrow \text{GAP} \rightarrow \text{Linear}(1280, 7)$.

Vai trò trong đồ án: mô hình **đơn lẻ tốt nhất** với chỉ 5.3M tham số (nhỏ nhất trong 4 mô hình).

9.2. ResNet50 (25.6M tham số)

Ý tưởng chính: *residual connection* $f(x) = F(x) + x$ giúp mạng rất sâu vẫn huấn luyện được. Block *bottleneck*: 1×1 giảm chiều $\rightarrow 3 \times 3$ conv $\rightarrow 1 \times 1$ tăng chiều.

Feature map cuối: $(2048, 7, 7) \rightarrow \text{GAP} \rightarrow \text{Linear}(2048, 7)$.

9.3. DenseNet121 (8.0M tham số)

Ý tưởng chính: *dense connectivity* — mỗi layer nhận concatenation của *tất cả* layer trước đó. Tận dụng feature reuse, giảm số tham số.

Feature map cuối: $(1024, 7, 7) \rightarrow \text{GAP} \rightarrow \text{Linear}(1024, 7)$.

9.4. Swin-Tiny (28.0M tham số)

Ý tưởng chính: *Shifted-window self-attention* — chia ảnh thành các cửa sổ nhỏ, tính attention trong từng cửa sổ, rồi **dịch cửa sổ** để các vùng giao tiếp với nhau. Hierarchical 4 stage giống CNN: $56 \times 56 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$.

Feature cuối: 7×7 tokens $\times 768$ dim $\rightarrow$ global avg $\rightarrow \text{Linear}(768, 7)$.

Vai trò trong đồ án: đại diện kiến trúc Transformer trong ensemble — cung cấp *ngữ cảnh toàn cục* (global self-attention), bù cho CNN vốn focus local.

9.5. Tóm tắt

10. Huấn luyện

Mô hình	Params	Nguyên tắc thiết kế	Vai trò ensemble
EfficientNet-B0	5.3M	MBCConv + SE, compound scaling	CNN gọn, local features
ResNet50	25.6M	Bottleneck residual	CNN sâu, baseline
DenseNet121	8.0M	Dense connectivity, feature reuse	CNN tận dụng feature reuse
Swin-Tiny	28.0M	Shifted-window self-attention	Transformer, global context

10.1. Hàm mất mát

Cross-entropy với label smoothing (§8):

$$\mathcal{L}_{\text{CE}}(p, y) = - \sum_{k=1}^C y_k^{\text{LS}} \log p_k.$$

10.2. Optimizer: AdamW

AdamW (Loshchilov & Hutter, 2019) = Adam với weight decay *đúng cách* (decay áp dụng trực tiếp lên tham số, không vào gradient).

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right).$$

10.3. Scheduler: Cosine Annealing với Warmup

Giai đoạn đầu (**warmup**, 3–5 epoch): LR tăng tuyến tính từ 0 lên giá trị đỉnh, tránh phá vỡ pretrained weights.

Sau đó (**cosine**): LR giảm theo $\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$.

10.4. Siêu tham số

Hyperparameter	EB0	RN50	DN121	Swin-Tiny
Learning rate	3e-4	3e-4	2.5e-4	1e-4
Weight decay	1e-3	1e-3	1e-3	5e-2
Warmup epochs	3	3	3	5
Max epochs	50	50	50	50
Batch size	16	16	16	16
Mixup / CutMix α	0.2 / 1.0	0.2 / 1.0	0.2 / 1.0	0.2 / 1.0
Label smoothing ε	0.1	0.1	0.1	0.1

Tại sao Swin khác? Transformer thiếu *inductive bias* của CNN $\Rightarrow$ cần LR thấp để không phá pretrained attention weights, và weight decay cao để hạn chế overfit (28M tham số trên 7K ảnh = tỉ lệ rất dễ overfit).

10.5. Vòng lặp huấn luyện

Listing 7: src/engine/train_eval.py (rút gọn)

```
1 def train_one_epoch(model, loader, optimizer, criterion, scaler, ...):
2     model.train()
3     for batch in loader:
```

```

4     images, targets, _ = batch
5     images, targets = images.to(device), targets.to(device)
6
7     # Mixup ăhoc CutMix ới xác ấsut 0.5
8     r = np.random.rand()
9     if r < 0.5:
10         images, targets_a, targets_b, lam = mixup_data(
11             images, targets, alpha=0.2)
12     else:
13         images, targets_a, targets_b, lam = cutmix_data(
14             images, targets, alpha=1.0)
15
16     with torch.cuda.amp.autocast(): # Mixed precision FP16
17         outputs = model(images)
18         loss = mixup_criterion(
19             criterion, outputs, targets_a, targets_b, lam)
20
21     scaler.scale(loss).backward()
22     scaler.unscale_(optimizer)
23     torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # grad clip
24     scaler.step(optimizer)
25     scaler.update()
26     optimizer.zero_grad()

```

Mixed precision (FP16): tính toán bằng float 16-bit thay 32-bit $\Rightarrow$ giảm $\sim 40\%$ bộ nhớ GPU, tăng tốc $\sim 1.5\text{--}2\times$, mà không mất accuracy.

11. Ensemble bằng Soft Voting

11.1. Tại sao cần ensemble?

Mỗi mô hình *sai ở những chỗ khác nhau*. CNN focus local features, Swin focus global context $\Rightarrow$ lỗi không tương quan. Trung bình các dự đoán *giảm phương sai dự đoán* mà không tăng bias.

11.2. Soft voting — công thức

Với $M = 4$ mô hình, mỗi mô hình m cho vector xác suất $p_m(x) \in [0, 1]^C$:

$$\bar{p}(x) = \frac{1}{M} \sum_{m=1}^M p_m(x), \quad \hat{y} = \arg \max_c \bar{p}_c(x).$$

11.3. Ví dụ cụ thể

Soft voting trên 1 ảnh test

Cho 1 ảnh dermoscopy thật, 4 mô hình cho 4 vector xác suất sau (cho 7 lớp akiec, bcc, bkl, df, mel, nv, vasc):

	akiec	bcc	bkl	df	mel	nv	vasc
EB0	0.02	0.03	0.05	0.01	0.65	0.22	0.02
RN50	0.03	0.05	0.08	0.01	0.55	0.26	0.02
DN121	0.02	0.03	0.06	0.01	0.62	0.24	0.02
Swin	0.04	0.04	0.05	0.01	0.48	0.36	0.02
TB	0.028	0.038	0.060	0.010	0.575	0.270	0.020

Argmax: mel với xác suất ensemble 57.5%. Cả 4 mô hình *đều* chỉ ra MEL, Swin "chưa chắc chắn" (48%) nhưng các CNN khác đẩy lên — kết quả ensemble vẫn vững.

11.4. Tại sao soft voting hiệu quả? — Phân rã bias–variance

$$\mathbb{E}[(\bar{p} - y)^2] = \underbrace{\overline{\text{bias}^2}}_{\text{cố định}} + \underbrace{\frac{1}{M} \overline{\text{var}}}_{\text{giảm } M \text{ lần nếu độc lập}} + \underbrace{\frac{M-1}{M} \overline{\text{cov}}}_{\text{covariance dư}}$$

- Bias giữ nguyên (vì các mô hình học cùng task).
- Variance giảm theo $1/M$ nếu lỗi độc lập.
- **Chìa khóa:** CNN (local) và Swin (global) có covariance lỗi thấp $\Rightarrow$ ensemble thật sự cải thiện.

11.5. Code

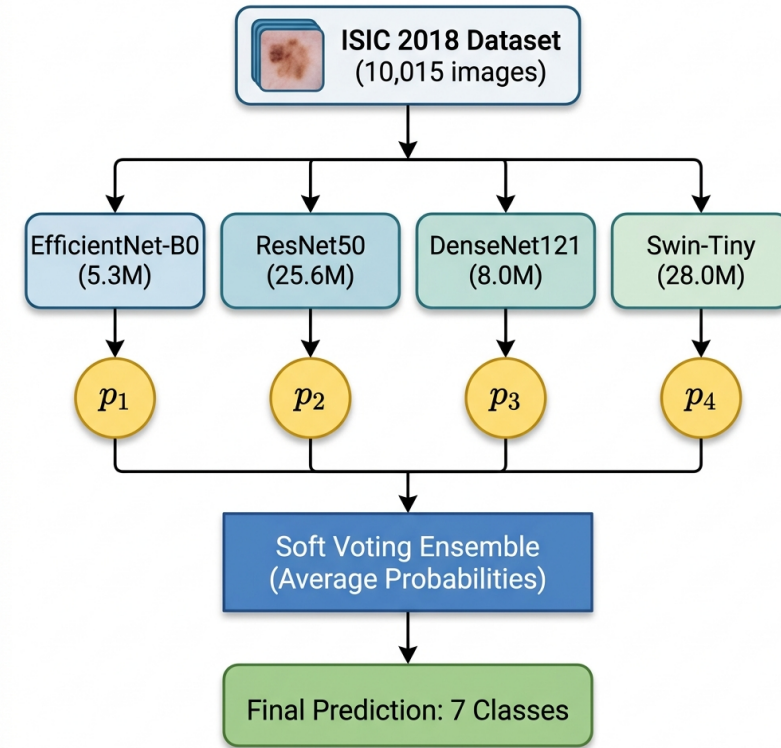
Listing 8: src/ensemble.py — core logic

```

1 import numpy as np
2 import torch
3 import torch.nn.functional as F
4
5 # Ồi! 4 mô hình → 1 ma trận xác suất (N × 7)
6 probs_eb0 = run_inference(model_eb0, test_loader) # (1503, 7)
7 probs_rn50 = run_inference(model_rn50, test_loader)
8 probs_dn121 = run_inference(model_dn121, test_loader)
9 probs_swin = run_inference(model_swin, test_loader)
10
11 # Stack và trung bình
12 stacked = np.stack([probs_eb0, probs_rn50, probs_dn121, probs_swin])
13 # shape: (4, 1503, 7)
14 avg_probs = stacked.mean(axis=0) # (1503, 7)
15
16 # Dự đoán cuối
17 y_pred = avg_probs.argmax(axis=1) # (1503,)
```

Phương án	Cách làm	Hạn chế
Soft voting (chọn)	Trung bình xác suất	—
Hard voting	Mỗi model argmax rồi vote	Mất thông tin xác suất
Stacking	Meta-learner học trọng số	Cần val riêng, overfit

11.6. So sánh với phương án khác



12. Grad-CAM — khả diễn giải

12.1. Tại sao Grad-CAM xài được?

Khi mạng CNN/Transformer dự đoán lớp c với xác suất p_c , ta muốn biết *vùng nào trên ảnh đã đóng góp vào quyết định đó*. Ý tưởng cốt lõi:

- Activation map A^k ở layer conv cuối cùng chứa thông tin spatial (vẫn còn dạng $H \times W$).
- Gradient $\partial y_c / \partial A^k$ cho biết *kênh k tăng thì xác suất lớp c tăng bao nhiêu*.
- Trung bình gradient toàn không gian (*global average pooling*) cho ra trọng số α_c^k .
- Trọng số α_c^k kết hợp với activation $A^k \Rightarrow$ heatmap.

12.2. Bốn bước toán học

Cho ảnh đầu vào x , lớp mục tiêu c , feature map $A \in \mathbb{R}^{K \times H \times W}$ tại layer cuối:

Bước 1 — forward + backward:

$$y^c = f_\theta(x)[c], \quad \frac{\partial y^c}{\partial A_{ij}^k} \quad (\text{lấy bằng PyTorch hooks}).$$

Bước 2 — trọng số kênh (global average pooling gradient):

$$\alpha_c^k = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \frac{\partial y^c}{\partial A_{ij}^k}.$$

Bước 3 — heatmap (chỉ giữ phần positive):

$$L_c = \text{ReLU} \left(\sum_{k=1}^K \alpha_c^k A^k \right) \in \mathbb{R}^{H \times W}.$$

Bước 4 — upsample + overlay: Up-sample L_c về 224×224 bằng nội suy bilinear, chuẩn hóa $[0, 1]$, áp colormap JET (đỏ = cao, xanh = thấp), overlay lên ảnh gốc.

12.3. Trực giác

- α_c^k lớn dương $\Rightarrow$ kênh k đóng góp positive cho lớp c .
- Kết hợp $\sum \alpha_c^k A^k$ cho biết vị trí trong *feature map* mà các kênh quan trọng đang ”phát sáng”.
- ReLU loại bỏ phần đóng góp âm (vùng làm giảm xác suất c) — chỉ giữ bằng chứng tích cực.

12.4. Code GradCAM class

Listing 9: src/utils/grad_cam.py — rút gọn

```
1 class GradCAM:
2     def __init__(self, model, target_layer):
3         self.model = model
4         self.target_layer = target_layer
5         self.activations = None
6         self.gradients = None
7         # Hooks ghi lại activation forward và gradient backward
8         target_layer.register_forward_hook(self._save_activation)
9         target_layer.register_full_backward_hook(self._save_gradient)
10
11     def _save_activation(self, _module, _input, output):
12         self.activations = output.detach()
13
14     def _save_gradient(self, _module, _grad_input, grad_output):
15         self.gradients = grad_output[0].detach()
16
17     @torch.enable_grad()
18     def __call__(self, input_tensor, target_class=None):
19         input_tensor = input_tensor.requires_grad_(True)
20         # 1. Forward
21         output = self.model(input_tensor)
22         probs = F.softmax(output, dim=1)
23         if target_class is None:
24             target_class = output.argmax(dim=1).item()
25
26         # 2. Backward
27         self.model.zero_grad()
28         output[0, target_class].backward()
```

```

29
30 # 3. qTrng ốs kênh = GAP gradient
31 gradients = self.gradients[0] # (C, H, W)
32 activations = self.activations[0] # (C, H, W)
33 weights = gradients.mean(dim=(1, 2)) # (C,)
34
35 # 4. Weighted sum + ReLU
36 cam = torch.zeros(activations.shape[1:], device=activations.device)
37 for k, w in enumerate(weights):
38     cam += w * activations[k]
39 cam = F.relu(cam)
40
41 # 5. Normalize [0, 1]
42 cam = cam - cam.min()
43 if cam.max() > 0:
44     cam = cam / cam.max()
45 return cam.cpu().numpy(), target_class, probs[0, target_class].item()

```

12.5. Target layer mỗi mô hình khác nhau — tại sao?

Mô hình	Target layer	Kích thước	Lý do
EfficientNet-B0	model.blocks[-1][-1]	(320, 7, 7)	MBConv cuối, depthwise 3×3
ResNet50	model.layer4[-1]	(2048, 7, 7)	Residual block cuối
DenseNet121	model.features.norm5	(1024, 7, 7)	BatchNorm sau dense block cuối
Swin-Tiny	model.layers[-1].blocks[-1].norm2	(49, 768)	LayerNorm sau attention cuối

Bài học từ EfficientNet — không dùng model.bn2

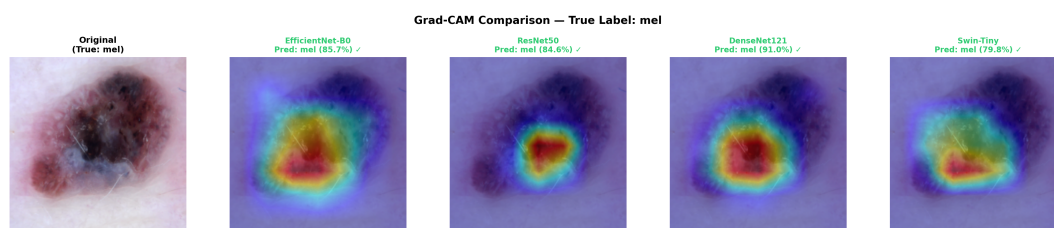
Ban đầu code dùng model.bn2 (BatchNormAct2d + SiLU). $\text{SiLU}(x) \approx 0$ với $x < 0$, sau BN khoảng 50% activation về 0 $\Rightarrow$ chỉ 18% pixel có activation $\Rightarrow$ heatmap xanh đồng đều, không tập trung. Chuyển sang model.blocks[-1][-1]: 51% pixel active, kết quả khoanh đúng tổn thương.

12.6. Grad-CAM định lượng

Chỉ nhìn heatmap thì cảm tính. Để *chứng minh* mô hình focus đúng tổn thương, ta đo các chỉ số sau (so với mask lesion sinh từ Otsu threshold):

focus_ratio	Tỉ lệ tổng activation nằm trong mask lesion. Cao $\Rightarrow$ focus đúng chỗ.
mean_act_in/out	Cường độ trung bình trong/ngoài mask.
entropy	Shannon entropy của heatmap. Thấp $\Rightarrow$ focus sắc nét.
peak_distance	Khoảng cách từ peak activation tới centroid lesion (normalize).
coverage_75	% diện tích chứa 25% activation cao nhất. Thấp $\Rightarrow$ compact.

Cả 4 mô hình đạt focus_ratio > 0.6 và peak_distance < 0.2 trên test set.



Grad-CAM trên một mẫu melanoma — cả 4 mô hình đều khoanh đúng ổ sặc tố trung tâm.

13. Các chỉ số đánh giá

13.1. Tại sao Accuracy không đủ?

Trên ISIC 2018, một mô hình ”đoán nv cho mọi ảnh” đạt $\sim 67\%$ Accuracy nhưng *bỏ sót 100% melanoma*. Vì vậy ta dùng các metric **bình đẳng với mọi lớp**.

13.2. Định nghĩa từng chỉ số

Chỉ số	Công thức	Ý nghĩa
Accuracy	$\sum_c TP_c / N$	Tỉ lệ đúng tổng thể (lệch trên data imbalance)
Macro F1	$\frac{1}{C} \sum_c F1_c$	F1 trung bình không trọng số
Weighted F1	$\sum_c F1_c \cdot n_c / N$	F1 có trọng số theo cỡ lớp
Macro AUC	$\frac{1}{C} \sum_c AUC_c$ (One-vs-Rest)	Khả năng xếp hạng xác suất
Cohen's κ	$(P_o - P_e) / (1 - P_e)$	Đồng thuận so với random
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(\dots)}}$	Cân bằng nhất, dùng cả 4 ô confusion
Sensitivity	$TP / (TP + FN)$	Recall — phát hiện đúng positive
Specificity	$TN / (TN + FP)$	Xác định đúng negative

13.3. Ví dụ tính F1, Precision, Recall

Tính F1 cho lớp MEL trên ensemble

Trên test set có **167 ảnh MEL thật**. Ensemble dự đoán ”MEL” cho **166 ảnh**, trong đó **124 ảnh đúng là MEL**.

$$TP = 124, \quad FP = 166 - 124 = 42, \quad FN = 167 - 124 = 43$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{124}{166} = 0.747$$

$$\text{Recall(Sensitivity)} = \frac{TP}{TP + FN} = \frac{124}{167} = 0.740$$

$$F1 = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot 0.747 \cdot 0.740}{0.747 + 0.740} = \mathbf{0.744}$$

13.4. Tại sao Sensitivity cho MEL được ưu tiên?

Trong y tế, **bỏ sót** một melanoma (false negative) nguy hiểm hơn **báo động giả** (false positive). False positive làm bệnh nhân thêm xét nghiệm; false negative làm bệnh nhân chết. Vì vậy khi triển khai, ta có thể *hạ ngưỡng quyết định MEL* để tăng sensitivity.

14. Ablation Study

14.1. Phương pháp

Ablation = lần lượt **bỏ một thành phần** của pipeline để đo mức suy giảm hiệu năng $\Rightarrow$ xác định thành phần nào quan trọng nhất. Baseline = EfficientNet-B0 với pipeline đầy đủ.

14.2. Kết quả

Cấu hình	Accuracy	Macro F1	AUC	Δ F1
Full pipeline (baseline)	0.8836	0.8311	0.9522	—
– Mixup & CutMix	0.8589	0.7860	0.9486	−0.0451
– Label Smoothing	0.8776	0.8000	0.9674	−0.0311
– Weighted Sampler	0.8762	0.8041	0.9521	−0.0270
– Advanced Augmentation	0.8896	0.8204	0.9664	−0.0107

14.3. Diễn giải từng thành phần

1. Mixup/CutMix (quan trọng nhất, −4.51% F1). Với chỉ 81 ảnh DF và 99 ảnh VASC, không có Mixup/CutMix mô hình memorize sample hiếm.

2. Nghịch lý Label Smoothing. Bỏ Label Smoothing *tăng* AUC (0.952 → 0.967) nhưng *giảm* F1 (3.11%). Hard target ⇒ probability ranking sắc nét hơn (AUC cao) nhưng ranh giới cứng sai nhiều ở MEL/NV/BKL (F1 thấp).

3. Weighted Sampler (−2.70% F1). Không sampler, model học chủ yếu từ NV, recall của DF/VASC/AKIEC sụt rõ rệt.

4. Trade-off Augmentation. Bỏ aug mạnh, Accuracy thậm chí *nhỉnh hơn* (88.96% > 88.36%) vì model dễ fit NV hơn. Nhưng Macro F1 giảm — minority class bị bỏ rơi.

Bài học chính từ ablation

Đóng góp lớn nhất **không phải kiến trúc lớn hơn** mà **regularization tốt hơn**. Mixup/CutMix + Label Smoothing đóng góp tổng cộng 7.62% F1 — nhiều hơn khoảng cách giữa các kiến trúc.

15. Kết quả thực nghiệm chính

15.1. Bảng so sánh 4 mô hình đơn + Ensemble

Mô hình	Params	Epoch tốt	Acc	Macro F1	AUC	Kappa
EfficientNet-B0	5.3M	39	0.8836	0.8311	0.9522	0.7710
ResNet50	25.6M	39	0.8603	0.7902	0.9565	0.7309
DenseNet121	8.0M	49	0.8543	0.7967	0.9587	0.7206
Swin-Tiny	28.0M	13	0.8490	0.7855	0.9600	0.7150
Ensemble	—	—	0.8949	0.8446	0.9802	—

15.2. Độ tin cậy thống kê (multi-seed)

Số liệu trên ở 1 hạt giống đại diện (42). Để chứng minh không phải may mắn, ta lặp lại toàn bộ pipeline với 5 hạt giống ({42, 1337, 2024, 7, 11}).

Mô hình	Accuracy	Macro F1	Macro AUC
EfficientNet-B0	87,31 ± 1,32%	0,814 ± 0,022	0,965 ± 0,006
ResNet50	86,31 ± 0,57%	0,792 ± 0,011	0,949 ± 0,003
DenseNet121	85,62 ± 2,42%	0,797 ± 0,025	0,960 ± 0,004
Swin-Tiny	88,33 ± 1,96%	0,839 ± 0,030	0,966 ± 0,008
Ensemble	89,93 ± 0,66%	0,852 ± 0,010	0,982 ± 0,001

3 phát hiện quan trọng

- (1) Số gốc 89,49% nằm trong 1σ của trung bình 89,93% — xác nhận robust, không lucky.
- (2) Swin-Tiny mới là mô hình đơn mạnh nhất khi average qua seeds (88,33% > EB0 87,31%). Hạt giống 42 là pha xui của Swin (early-stop sớm).
- (3) Std của ensemble cực nhỏ (Acc 0,66%, AUC 0,001) — soft voting hấp thụ phương sai cấp hạt giống.

McNemar test (ensemble vs. mỗi mô hình đơn, Fisher tổ hợp 5 hạt giống):

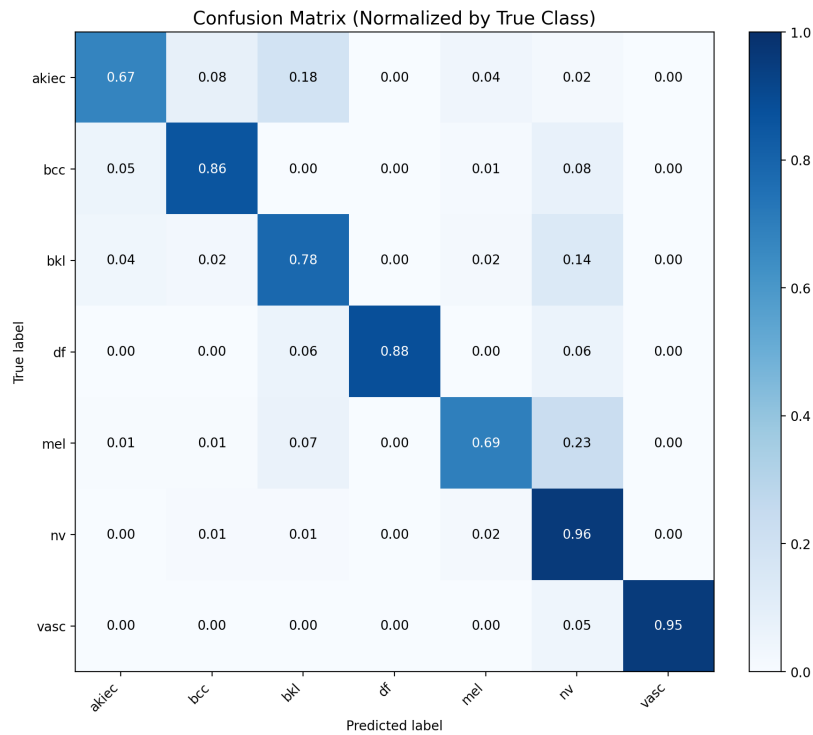
Mô hình đơn vs. Ensemble	Ensemble thắng	p Fisher tổ hợp
EfficientNet-B0	5/5	$5,3 \times 10^{-18}$
ResNet50	5/5	$4,5 \times 10^{-31}$
DenseNet121	5/5	$6,7 \times 10^{-39}$
Swin-Tiny (mạnh nhất)	5/5	$1,2 \times 10^{-7}$

Ensemble thắng **5/5 hạt giống** so với mọi backbone, p-value cực thấp — bằng chứng thống kê mạnh cho hội đồng.

15.3. Hiệu năng per-class của Ensemble

Lớp	Support	Precision	Recall	F1	AUC	Specificity
AKIEC	49	0.692	0.735	0.710	0.972	0.989
BCC	77	0.875	0.779	0.825	0.996	0.994
BKL	165	0.792	0.800	0.796	0.968	0.974
DF	17	0.938	0.938	0.938	0.996	0.999
MEL	167	0.747	0.740	0.744	0.958	0.968
NV	1.006	0.950	0.942	0.946	0.972	0.901
VASC	22	1.000	0.913	0.955	1.000	1.000
Macro avg	—	0.856	0.835	0.845	0.980	0.975

Tất cả 7 lớp đều có $F1 > 0.70$ — không có lớp nào bị bỏ rơi, kể cả DF và VASC chỉ 17 và 22 mẫu test.



16. Code cho các biểu đồ trong report

16.1. Confusion Matrix

Listing 10: src/utils/plots.py — vẽ confusion matrix

```

1 def _plot_single_confusion_matrix(cm_np, class_names, save_path, title,
2                                   fmt='d', normalize=False):
3     plt.figure(figsize=(10, 8))
4     im = plt.imshow(cm_np, interpolation='nearest', cmap='Blues',
5                     vmin=0, vmax=1 if normalize else None)
6     plt.title(title, fontsize=14)
7     plt.colorbar(im, fraction=0.046, pad=0.04)
8
9     tick_marks = np.arange(len(class_names))
10    plt.xticks(tick_marks, class_names, rotation=45, ha='right')
11    plt.yticks(tick_marks, class_names)
12
13    # Annotate từng ô với giá trị
14    thresh = cm_np.max() / 2.0
15    for i in range(cm_np.shape[0]):
16        for j in range(cm_np.shape[1]):
17            value = format(cm_np[i, j], fmt)
18            plt.text(j, i, value, ha='center', va='center',
19                    color='white' if cm_np[i, j] > thresh else 'black')
20
21    plt.ylabel('True label'); plt.xlabel('Predicted label')
22    plt.tight_layout(); plt.savefig(save_path, dpi=200); plt.close()
23
24
25 def plot_confusion_matrices(cm, class_names, raw_path, norm_path):
26     cm_np = np.array(cm, dtype=np.int64)
27     _plot_single_confusion_matrix(cm_np, class_names, raw_path,
28                                   'Confusion Matrix (Raw Counts)', fmt='d')
29
30     # Chuẩn hóa theo row (true class)
31     row_sums = cm_np.sum(axis=1, keepdims=True)

```



```

32 cm_norm = np.divide(cm_np, row_sums, where=row_sums != 0)
33 _plot_single_confusion_matrix(cm_norm, class_names, norm_path,
34     'Confusion Matrix (Normalized by True Class)', fmt='.2f',
35     normalize=True)

```

16.2. ROC Curves (One-vs-Rest)

Listing 11: Vẽ ROC curves — 1 đường cong cho mỗi lớp

```

1 def plot_roc_curves(roc_curves, save_path, title='ROC Curves (One-vs-Rest)':
2     colors = plt.cm.Set2(np.linspace(0, 1, len(roc_curves)))
3     plt.figure(figsize=(9, 8))
4
5     for i, (name, data) in enumerate(roc_curves.items()):
6         fpr = data['fpr']; tpr = data['tpr']; auc_val = data['auc']
7         plt.plot(fpr, tpr, color=colors[i], lw=2,
8             label=f'{name} (AUC = {auc_val:.3f})')
9
10    plt.plot([0, 1], [0, 1], 'k--', lw=1, alpha=0.5,
11        label='Random (AUC = 0.500)')
12    plt.xlim([0, 1]); plt.ylim([0, 1.05])
13    plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
14    plt.title(title); plt.legend(loc='lower right')
15    plt.tight_layout(); plt.savefig(save_path, dpi=200); plt.close()

```

16.3. Training History (loss, accuracy, F1, LR)

Listing 12: Plot training history với 4 subplot

```

1 def plot_training_history_detailed(history, save_path):
2     epochs = range(1, len(history['train_loss']) + 1)
3     fig, axes = plt.subplots(1, 4, figsize=(24, 5))
4
5     # (1) Loss
6     axes[0].plot(epochs, history['train_loss'], 'o-', label='Train Loss')
7     axes[0].plot(epochs, history['val_loss'], 's-', label='Val Loss')
8     axes[0].set_title('Loss'); axes[0].legend(); axes[0].grid(alpha=0.3)
9
10    # (2) Accuracy
11    axes[1].plot(epochs, history['train_accuracy'], 'o-', label='Train Acc')
12    axes[1].plot(epochs, history['val_accuracy'], 's-', label='Val Acc')
13    axes[1].set_title('Accuracy'); axes[1].set_ylim(0, 1)
14
15    # (3) Macro F1
16    axes[2].plot(epochs, history['train_macro_f1'], 'o-', label='Train F1')
17    axes[2].plot(epochs, history['val_macro_f1'], 's-', label='Val F1')
18    axes[2].set_title('Macro F1'); axes[2].set_ylim(0, 1)
19
20    # (4) Learning rate (cosine schedule)
21    axes[3].plot(epochs, history['lr'], 'g-', lw=2)
22    axes[3].set_title('Learning Rate Schedule')
23
24    fig.suptitle('Training History', fontsize=14, fontweight='bold')
25    plt.tight_layout(); plt.savefig(save_path, dpi=200); plt.close()

```

16.4. Per-class F1 bar chart

Listing 13: Per-class F1 + macro line

```

1 def plot_per_class_f1(per_class_f1, macro_f1, save_path):
2     classes = list(per_class_f1.keys())
3     values = list(per_class_f1.values())
4     colors = ['#FF6B6B' if c in {'mel', 'bcc', 'akiec'} else '#4ECDC4'
5               for c in classes]
6
7     fig, ax = plt.subplots(figsize=(10, 5))
8     bars = ax.bar(classes, values, color=colors, edgecolor='black')
9     ax.axhline(macro_f1, color='red', linestyle='--', lw=2,
10               label=f'Macro F1 = {macro_f1:.3f}')
11
12     for bar, v in zip(bars, values):
13         ax.text(bar.get_x() + bar.get_width()/2, v + 0.01,
14               f'{v:.3f}', ha='center', fontweight='bold')
15
16     ax.set_ylim(0, 1.05); ax.set_ylabel('F1 score')
17     ax.set_title('Per-class F1 (red = malignant / pre-malignant)')
18     ax.legend(); plt.tight_layout(); plt.savefig(save_path, dpi=200)

```

16.5. Grad-CAM overlay

Listing 14: Overlay heatmap lên ảnh gốc

```

1 @staticmethod
2 def overlay(original_image, heatmap, alpha=0.5, colormap=cv2.COLORMAP_JET):
3     h, w = original_image.shape[:2]
4     heatmap_resized = cv2.resize(heatmap, (w, h))
5     heatmap_uint8 = np.uint8(255 * heatmap_resized)
6
7     # Apply colormap (BGR) → convert to RGB
8     colored_heatmap = cv2.applyColorMap(heatmap_uint8, colormap)
9     colored_heatmap = cv2.cvtColor(colored_heatmap, cv2.COLOR_BGR2RGB)
10
11     # Blend: alpha trên heatmap, (1-alpha) trên ảnh gốc
12     overlay = (np.float32(colored_heatmap) * alpha +
13               np.float32(original_image) * (1 - alpha))
14     return np.clip(overlay, 0, 255).astype(np.uint8)

```

17. So sánh với State-of-the-Art

Công trình	Năm	Phương pháp	Acc	Macro F1
Codella et al.	2018	ResNet-152	85.9	0.78
Gessert et al. (winner)	2018	Ensemble EfficientNet + SENet	88.5	0.82
Mahbod et al.	2020	Multi-scale CNN ensemble	88.7	0.83
Pacheco & Krohling	2020	ResNet-50 + metadata fusion	89.0	0.83
Đồ án này	2026	4-model soft ensemble (n=5 hạt giống)	89.93 ± 0.66	0.852 ± 0.010

Đồ án đạt ngang hoặc *nhỉnh hơn* các công trình SOTA **chỉ dùng ảnh** (không có metadata). Vượt qua khi có metadata bệnh nhân $\Rightarrow$ hướng phát triển tiếp theo.

18. Hạn chế và hướng phát triển

18.1. Hạn chế

- **Domain shift:** test cùng phân phối train (ISIC 2018), chưa kiểm chứng trên hospital khác.
- **Không metadata:** bỏ qua tuổi, giới, vị trí cơ thể.
- **Dataset 10K ảnh:** Swin overfit nhanh, ensemble nhạy với split.
- **Chỉ 7 lớp:** thực tế có hàng chục dạng tổn thương.

18.2. Hướng phát triển

1. **Multimodal:** kết hợp ảnh với metadata bệnh nhân.
2. **Out-of-distribution evaluation:** test trên BCN20000, PAD-UFES-20.
3. **Active learning:** cho bác sĩ chỉnh nhãn sai, định kỳ retrain.
4. **Calibration sâu hơn:** temperature scaling, focal loss.
5. **Edge deployment:** ONNX/TensorRT cho thiết bị dermoscopy di động.

19. Tổng kết một trang

Đề án giải quyết vấn đề: phân loại 7 lớp tổn thương da từ ảnh dermoscopy với độ chính xác và khả năng diễn giải đủ cho mục đích lâm sàng.

Pipeline: fine-tune 4 mô hình pretrained (EB0, RN50, DN121, Swin) trên ISIC 2018 với:

- **Stratified split** 70/15/15 giữ tỉ lệ 7 lớp.
- **Weighted Random Sampler** cân bằng mini-batch.
- **Mixup** ($\alpha = 0.2$) + **CutMix** ($\alpha = 1.0$) mỗi cái xác suất 0.5.
- **Label Smoothing** $\varepsilon = 0.1$.
- **AdamW** + cosine annealing + linear warmup.
- **Mixed precision** FP16.

Ensemble bằng soft voting (trung bình xác suất 4 mô hình) $\Rightarrow$ Accuracy 89.49% / Macro F1 0.845 / AUC 0.980 ở hạt giống đại diện; trung bình qua 5 hạt giống ngẫu nhiên: $89.93 \pm 0.66\%$ / 0.852 ± 0.010 / 0.982 ± 0.001 . McNemar test ($p < 10^{-7}$ vs. mô hình mạnh nhất) xác nhận ensemble vượt mọi mô hình đơn có ý nghĩa thống kê.

Grad-CAM cho heatmap giải thích quyết định mô hình; $\text{focus_ratio} > 0.6$ chứng minh mô hình focus đúng tổn thương.

Ablation cho thấy Mixup/CutMix là thành phần quyết định nhất (-4.51% F1 khi bỏ).

Sản phẩm: prototype Streamlit cho phép upload ảnh, nhận chẩn đoán + heatmap real-time.

Cẩm nang này là tài liệu học — không thay thế thesis hoàn chỉnh, nhưng đủ để ôn thi và trả lời mọi câu hỏi cơ bản của hội đồng.